ENGINEERING REPORT · PPG-BASED SPO2 ESTIMATION

SpO2 Encoder-Decoder Pipeline

How `train2.py` pretrains a PPG waveform encoder by self-supervised reconstruction, transplants it into a physics-guided residual estimator, and what four rounds of leave-one-subject-out validation found.

Files train1.py → train2.py **Dataset** 11 subjects · RValues_RawPPG_Aref_
Signal RED / IR PPG, 400-sample windows **Date** 2026-09-01

BOTTOM LINE

The pipeline works — the encoder pretrains cleanly, freezing is genuinely frozen, and the residual head trains stably. But across a proper 11-fold leave-one-subject-out evaluation, the learned correction does not reliably beat the classical ratio-of-ratios physics baseline (SpO2 = a + b·R). The best configuration found gets within **0.8%** of the baseline's mean RMSE and wins on **6 of 11** held-out subjects — competitive, not a clear win. Two real bugs were found and fixed along the way; the remaining gap looks like a data-quantity ceiling rather than a tunable one.

01 · ARCHITECTURE

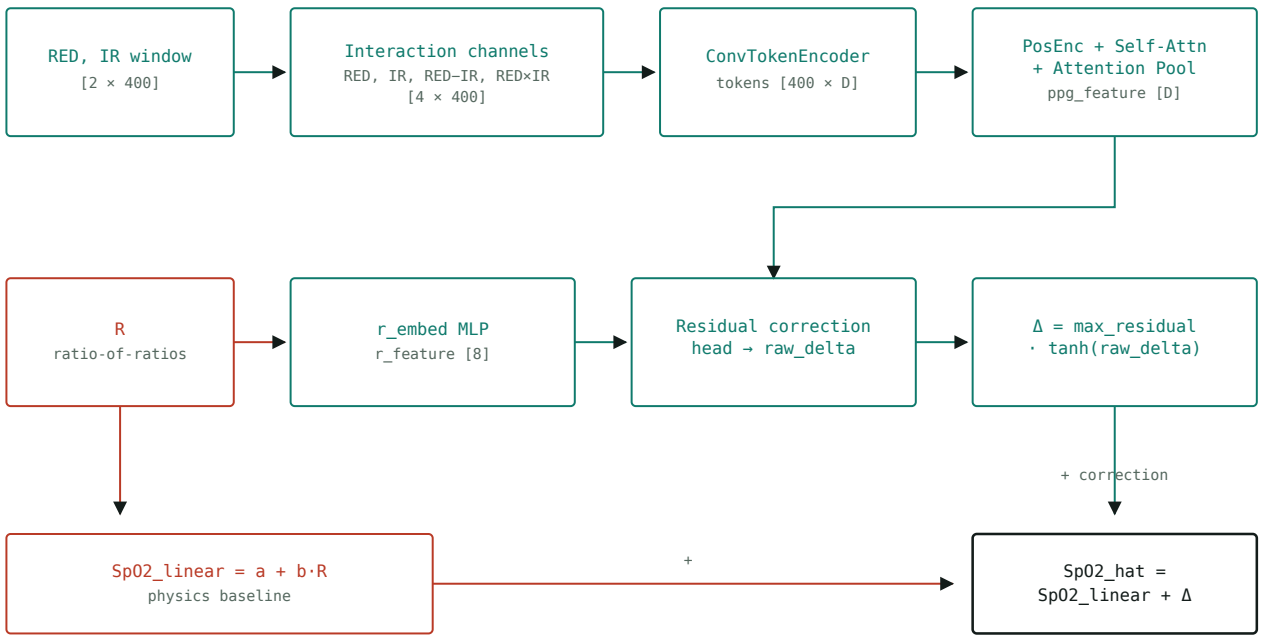
A physics baseline the network is only allowed to nudge

Every window is 400 samples (16 s at 25 Hz) of synchronized RED and IR PPG, paired with a ratio-of-ratios value `R` and a reference `SpO2_Rad` label. The model doesn't predict SpO2 from scratch — it starts from the classical pulse-oximetry formula `SpO2_linear = a +`

`b·R`, fit once per fold by least squares on training subjects only, and learns a small, bounded correction on top of it:

```
SpO2_hat = SpO2_linear + max_residual · tanh(correction_head(ppg_feature, r_feature))
```

The correction head's last layer is zero-initialized, so at the start of training the network reproduces the physics baseline exactly and can only earn deviation from it through gradient descent.



Forward pass. Teal marks the learned path (encoder, attention, correction head); red marks the fixed physics path. The two meet only at the final sum — the network can shift the prediction but never replace the baseline outright.

MODULE REFERENCE

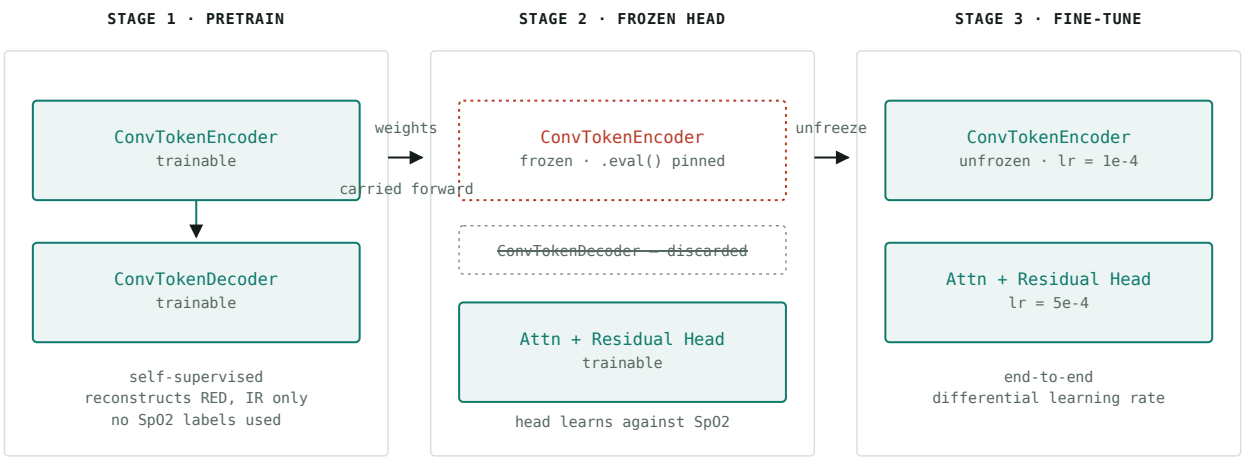
MODULE	ROLE	OUTPUT SHAPE
ConvTokenEncoder	3× Conv1d (k=7,5,3), no temporal downsampling	`[400, D]`
LearnablePositionalEncoding	Learned additive position embedding	`[400, D]`
TemporalSelfAttentionBlock	Multi-head self-attention + FFN, post-norm	`[400, D]`
AttentionPooling	Learned softmax weighting over the 400 tokens	`[D]`

MODULE	ROLE	OUTPUT SHAPE
r_embed	2-layer MLP on the scalar ratio R	[8]
correction_head	MLP, zero-initialized final layer	[1]

02 · ARCHITECTURE

Three training stages share one encoder

The ConvTokenEncoder above is never trained against SpO2 labels directly. It is pretrained on raw signal reconstruction first, then transplanted into the full model and frozen, then finally unfrozen for a gentler joint fine-tune. This is the encoder-decoder idea the report is named for: pretrain an encoder with a decoder attached, discard the decoder, and drive an estimator head from the encoder's learned representation.



Same encoder box, three states. Stage 1 trains it against its own reconstruction; Stage 2 freezes it and discards the decoder; Stage 3 unfreezes it at a tenth of the head's learning rate.

STAGE CONFIGURATION

STAGE	OBJECTIVE	LEARNING RATE	EPOCHS
1 · Pretrain	MSE reconstruction of RED, IR	1e-3	150
2 · Frozen head	Huber loss vs. SpO2, encoder frozen	1e-3	60

STAGE	OBJECTIVE	LEARNING RATE	EPOCHS
3 · Fine-tune	Huber loss vs. SpO2, end-to-end	1e-4 / 5e-4	60

03 · PROCESS

What went wrong first, and how each fix was found

None of the above worked on the first attempt. Four distinct problems surfaced in sequence, each diagnosed from evidence in the training logs rather than guessed at.

1 · Freezing didn't actually freeze

The first version of the pretrained-encoder pattern (built earlier, in a companion script) set `requires_grad = False` on the encoder's parameters and called it frozen. It wasn't: `ConvTokenEncoder` uses `BatchNorm1d`, whose running statistics are *buffers*, not parameters — they keep drifting under `model.train()` regardless of `requires_grad`, because BatchNorm normalizes with per-batch statistics whenever a module is in train mode. `PretrainedRawSelfAttentionSpO2Net` was built with the fix in from the start: `set_encoder_trainable()` also calls `.train(trainable)` on the encoder, and a `train()` override re-pins it to `.eval()` even when the parent model is switched back to train mode.

2 · The reconstruction loss exploded

The first Stage 1 run produced a validation MSE in the **600,000–700,000** range and never converged — worse than the ~302 MSE of just predicting each channel's mean. The cause was the interaction channels themselves. Measured on real subject data:

PER-CHANNEL VARIANCE, SUBJECT8

CHANNEL	STD	VARIANCE
RED	13.57	184
IR	20.48	419
RED − IR	9.39	88

CHANNEL	STD	VARIANCE
RED × IR	**352.66**	**124,371**

RED×IR carries roughly **300–600×** the variance of the other three channels. The autoencoder's decoder was asked to reconstruct all four channels under one unweighted MSE, so nearly all of the loss and the gradient came from that one derived, redundant channel — RED−IR and RED×IR are both deterministic functions of RED and IR, so reconstructing them added nothing but distorted the objective. The fix: keep the encoder's input at 4 channels (it needs to see what it will see downstream), but have the decoder reconstruct only the 2 true underlying signals, RED and IR. Validation MSE then converged normally, bottoming out around **65–70** — roughly 4.5× better than the mean-prediction baseline.

3 · A single train/val/test split lied

With the reconstruction bug fixed, one deterministic split (`val=Subject11` , `test=Subject2`) looked like a clear win: fine-tuned validation RMSE **4.56**, beating the physics baseline's **5.19**. But on the held-out test subject, fine-tuned RMSE was **6.93** — worse than the baseline's **6.25**. With only 11 subjects, a single validation subject is a high-variance signal: it can reward a correction that fits that one subject without telling you anything about the next one. This motivated moving to leave-one-subject-out (LOSO) cross-validation for every result that follows.

4 · Four LOSO rounds, chasing the gap

Each round re-pretrains Stage 1 from scratch per fold, using only that fold's training and validation windows — the held-out test subject's raw signal never touches the encoder, labeled or not.

LOSO ROUNDS · MEAN TEST RMSE ACROSS 11 SUBJECTS

ROUND	CHANGE FROM PREVIOUS	LINEAR	FINE-TUNED	WINS
1	Baseline · hidden_dim=32, 1 val subject	3.191	3.295	4 / 11
2	Tighter output: max_residual 5→2, λ 0.001→0.01	3.191	3.281	5 / 11
3	Smaller model: hidden_dim 32→16, heads 4→2	3.191	3.216	6 / 11
4	2 validation subjects instead of 1	3.047	3.141	6 / 11

Round 3's capacity cut moved the gap roughly 3× more than round 2's output regularization did — the model had more parameters than ~200 training windows could constrain, and shrinking it (14,394 vs. 34,842 trainable weights) was a bigger lever than penalizing the correction's magnitude. Round 4 tested a specific, evidence-backed hypothesis: on the two losing folds in round 3, validation RMSE tracked *below* the physics baseline for the entire run while test RMSE ended up above it — the correction was fitting that one validation subject's bias, not a generalizable pattern. Averaging checkpoint selection over 2 validation subjects fixed that specific fold (Subject3 flipped from a loss to a win) but introduced two new regressions elsewhere (Subject5, Subject8), leaving the aggregate gap roughly where it started. That plateau across two independent fixes is itself informative: it points at a ceiling set by having 11 subjects, not at a remaining bug.

Best configuration, per subject

Round 3 (shrunk model, 1 validation subject) was the closest to the physics baseline. Per-fold detail:

ROUND 3 · RMSE BY HELD-OUT TEST SUBJECT

TEST SUBJECT	LINEAR	FROZEN	FINE-TUNED	VS. BASELINE
Subject1	3.871	4.117	4.371	● loses
Subject2	6.653	6.717	6.748	● loses
Subject3	1.995	2.471	2.434	● loses
Subject4	4.257	4.255	4.249	● wins
Subject5	2.183	2.184	2.185	● loses
Subject6	2.309	2.750	2.587	● loses
Subject7	2.845	2.682	2.599	● wins
Subject8	1.148	0.998	1.003	● wins
Subject9	2.451	2.072	2.018	● wins
Subject11	5.586	5.470	5.444	● wins
Subject12	1.807	1.765	1.737	● wins

The wins and losses aren't clustered by any obvious property (recording length, R range) — consistent with subject-level noise rather than a systematic model failure the architecture

could still be fixed to avoid.

04 · CONCLUSIONS

What to take from this

- **The pipeline is correct.** Encoder pretraining converges to a healthy reconstruction loss, freezing genuinely holds the encoder fixed (including its BatchNorm statistics), and fine-tuning uses a sensible differential learning rate.
- **It does not yet beat the physics baseline.** The best of four tuning rounds lands within 0.8% of the linear ratio-of-ratios RMSE and wins on a bare majority of subjects — a wash, not a victory.
- **Capacity mattered more than output regularization.** Halving `hidden_dim` closed more of the gap than tightening `max_residual` and `residual_lambda` did, for ~200 total training windows.
- **Single-subject validation is not a safe early-stopping signal at this scale.** Adding a second validation subject fixed one diagnosed failure mode but surfaced others of similar size — the fixes trade places more than they compound.
- **The realistic next lever is more subjects, not more tuning.** Two independent capacity/regularization interventions plateaued at the same win rate; that is the signature of a data ceiling.

`train1.py · train2.py · enc_dec.py`

`Checkpoints: loso_checkpoints/ (4 rounds preserved under sibling directories)`